

LinkedList

¿Qué es una LinkedList?

Una LinkedList es una simple cadena de nodos conectados. Podemos imaginarla como un tren: cada vagón contiene un dato (por ejemplo, un número) y está unido al siguiente mediante un gancho. A ese vagón junto con su gancho se le llama nodo. Una LinkedList es, por tanto, un conjunto de nodos conectados en cadena.

Los elementos se van guardando uno detrás de otro. El último elemento que añadas se coloca siempre al final.

Diferencias con un Array

Aunque LinkedList y Array sirven para almacenar colecciones de datos, presentan diferencias importantes:

- El array tiene un tamaño fijo que se determina al crearlo. La LinkedList no tiene un tamaño predefinido y crece de forma dinámica.
- Si quieres modificar algo al principio de un array, hay que mover todos los elementos. En LinkedList esto no ocurre, ya que los nodos no tienen posiciones fijas en memoria, sino que están enlazados en cadena.
- El acceso a elementos es más lento en LinkedList. Por ejemplo, para acceder al elemento número 20, hay que recorrer primero los 19 anteriores, ya que no existe acceso directo por índice como en un array.

Lo recomendable es que si necesitas acceder frecuentemente por índice a los elementos de la lista, uses ArrayList, pero si lo que necesitas es ir borrando frecuentemente al principio de la lista o al final, como en el ejemplo que se mostrará a continuación, uses LinkedList.

Interfaces Implementadas

La clase LinkedList implementa las siguientes interfaces:

- Serializable
- Cloneable
- Iterable
- Collection
- Deque
- List
- Queue

Métodos más usados

A continuación, estarán los métodos que más se suelen utilizar con LinkedList, junto a algunos ejemplos prácticos, creando un programa funcional con algunos de estos métodos, los que se consideran los más importantes.

Partiendo de esto:

```
int op, addSong;
String borrarSong, findSong;

LinkedList<Cancion> lista = new LinkedList<>();

Cancion c1 = new Cancion("ASÚMELO",3.00,"Astra");
Cancion c2 = new Cancion("Chansey",2.45,"Astra");
Cancion c3 = new Cancion("Ángel guapo",3.15,"Mario");

Playlist pl1 = new Playlist(lista);

System.out.println("Elige una opción");
op=Leer.datoInt();
```

Elaboramos un programa que gestione una playlist de canciones.

Métodos de inserción

- **add(dato)** – añade el elemento al final de la lista.

```
public void agregarCancion(Cancion c) {

    lista.add(c);

}
```

Creamos un método para añadir canciones a la playlist, pasando una canción por parámetros.

- **addFirst(dato)** – añade el elemento al principio de la lista.
- **addLast(dato)** – añade el elemento al final, igual que add().
- **add(int index, Object o)** – añade el elemento en la posición index indicada.

Métodos de eliminación

- **removeFirst()** – elimina el primer elemento y lo devuelve.

```
public void borrarPrimera() {  
    lista.removeFirst();  
}
```

Esto nos servirá para borrar la canción de la playlist y que empiece a sonar, ya que si se borra de la cola, puede ser por dos motivos, o porque no la queremos escuchar y la borramos manualmente, o porque ha empezado a sonar y se ha borrado automáticamente. En nuestro caso, puede servir para ambas cosas ya que el programa no está tan desarrollado.

- **removeLast()** – elimina el último elemento y lo devuelve.
- **remove(dato)** – elimina la primera aparición de ese valor.

```
public void borrarCancion(String nombre) {  
    Cancion c = findByNombre(nombre);  
    if(c.getNombre().equalsIgnoreCase(nombre)) {  
        lista.remove(c);  
    }  
}
```

Este método nos sirve para borrar la canción de la cola que no queramos escuchar. Se busca por nombre.

- **remove(int index)** – elimina el elemento que se encuentre en esa posición.

Métodos de acceso

- **getFirst()** – devuelve el primer elemento de la lista.

```
public void obtenerSiguiente() {  
    System.out.println(lista.getFirst());  
}
```

Esto nos sirve para ver la siguiente canción que sonará.

- **getLast()** – devuelve el último elemento de la lista.
- **get(int index)** – devuelve el elemento de la posición index indicada.

Métodos de utilidad

- **size()** – devuelve el número de elementos que tiene la lista.
- **isEmpty()** – devuelve un booleano indicando si la lista está vacía o no.

```
public void mostrarCola() {  
    Iterator <Cancion> it = lista.iterator();  
    while(it.hasNext() && !lista.isEmpty()) {  
        Cancion c = it.next();  
        System.out.println("Cola: "+c);  
    }  
}
```

Este método sirve para mostrar la cola. Lo hacemos con un iterator ya que nos proporciona algunas ventajas respecto a los otros bucles. Usamos el método isEmpty() para comprobar que la lista no está vacía. Este método devuelve true si la lista está vacía, por lo tanto, metemos en el while que si es distinto de true, que pare el bucle, junto al método hasNext(). Quizás no es el mejor ejemplo, ya que en iterator, con llamar en el while al método hasNext, es suficiente para que funcione, pero lo pongo para que se entienda de qué va el metodo “isEmpty()”.

Como ampliación, es bueno recalcar que para recorrer de buena forma una LinkedList, usar un iterator es buena idea para ir de extremo a extremo de la lista, aquí tenemos un ejemplo.

```
public void recorrerListaEjemplo() {  
    Iterator<Cancion> it = lista.iterator();  
    while (it.hasNext()) {  
        Cancion c = it.next();  
        System.out.println(c);  
    }  
}
```

Declaramos el iterador y guardamos en la variable “it” el método “iterator” que recorre la lista.

En el while decimos que mientras que it tenga siguiente, se ejecute lo que hay dentro de las llaves, que en este caso es un syso imprimiendo las canciones que hay en la lista.

Caso de uso reales

Historial de google: Cada página sería un nodo, y el botón de atrás sería el que va recorriendo la lista conforme pulsas.

Lista de reproducción: conforme vayas pasando las canciones, va recorriendo la lista. Cuando pones una canción en cola, se añade al final.